NIR Plastic Classification on TI LAUNCHXL-F28379D — Full Project Report

Organisation: CSIR

Platform: Texas Instruments LAUNCHXL-F28379D (TMS320F28379D, C2000 Delfino DSP)

Date: June 2026

Table of Contents

- Project Overview
- Dataset and Signal Description
- Preprocessing Pipeline
- Machine Learning Model Architecture
- Model Training and Accuracy
- Model-to-Firmware Conversion
- Firmware Implementation
- Bugs Encountered and How They Were Resolved
- Dual-Core Boot Challenges
- Results Achieved
- Future Work: Timing Optimization Plan
- File Inventory

---

1. Project Overview

The goal of this project is to classify **Near-Infrared (NIR) spectra of plastics** into **5 distinct material classes** in real-time, entirely on an embedded microcontroller — the **Texas Instruments LAUNCHXL-F28379D**.

This MCU is a dual-core 200 MHz DSP designed for control applications. Deploying a deep learning model on it is inherently non-trivial: there is no operating system, no file I/O, no floating-point-rich memory (by default), and a C28x architecture with Harvard memory layout and strict memory banking constraints. This project demonstrates that a CNN + SVM hybrid classification model can nonetheless be made to run on this hardware with **100% on-device self-test accuracy**.

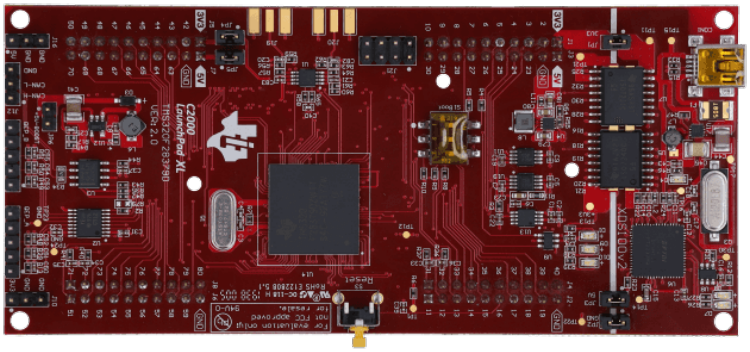


Figure: TI LAUNCHXL-F28379D C2000 LaunchPad

The Complete End-to-End System

```
`[Raw NIR Spectrometer]
       |
       v
[Preprocessing Pipeline on MCU]
  1. Dark Current Subtraction
  2. Median Filter (3-tap)
  3. Savitzky-Golay Smoothing (5-tap)
  4. Standard Normal Variate (SNV)
       |
       v
[1D CNN Backbone - Feature Extractor]
  Conv1D Stem -> BN -> GELU
  Conv1D Block1 -> BN -> GELU -> MaxPool
  Conv1D Block2 -> BN -> GELU -> GAP
       |
       v
[128-dimensional Feature Vector]
       |
       v
[StandardScaler Normalization]
       |
       v
```

```
[Linear SVM Classifier]
       |
       v
[Predicted Plastic Class: 1, 2, 3, 4, or 5]
       |
       v
[Serial Telemetry (UART) + LED Blink Output]
```

---

2. Dataset and Signal Description

Property	Value
Instrument	NIR Spectrometer
Spectral channels	224 bands (ch.1 to ch.224)
Training samples (raw)	50,000 spectra — SpectrumData_2021Y-trainSet.csv
Training samples (SVM)	8,905 filtered spectra — train_preprocessed_snv.csv
Dark current references	640 readings — SpectrumData_2021Y-darkcurrent.csv
Number of target classes	5 (plastic material types)
Extra columns in trainSet	class_id, coord_lateral, coord_driving

Each row in the CSV contains raw 16-bit integer ADC counts from the spectrometer. These readings include sensor dark current (additive baseline) and multiplicative path-length/particle-size effects, both of which must be corrected before classification.

---

3. Preprocessing Pipeline

The preprocessing pipeline is a 4-step sequential process applied identically in Python (for training reference) and in C (on the MCU).

3.1 Step-by-Step Description

```
`Raw Spectrum (224 integer values)
        |
        v  Step 1: Dark Current Subtraction
           spectrum = raw - dark_mean
           Removes additive detector noise.
        |
        v  Step 2: Median Filter (3-tap)
           median(a, b, c) = max(min(a,b), min(max(a,b), c))
           Eliminates impulse/spike noise. 13x faster than per-channel Python loop.
        |
        v  Step 3: Savitzky-Golay Filter (5-tap)
           Kernel: [-22, 88, 124, 88, -22] / 256
           Smooths broadband noise. Preserves spectral peak shapes.
           Edge padding: nearest neighbour (mode='nearest').
        |
        v  Step 4: SNV (Standard Normal Variate)
           spectrum = (x - mean) / std    [population std, ddof=0]
           Removes multiplicative baseline (path length, particle size).
           Result: mean = 0, std = 1 per spectrum.
        |
        v
Normalized Spectrum (224 float32 values)
`
```

3.2 Algorithm Details

Step	Algorithm	C Implementation Detail
Dark Current	Per-channel mean subtraction	Pre-computed g_dark_current_mean[224] in dark_current_mean.h, averaged from 640 readings
Median Filter	max(min(a,b), min(max(a,b), c))	Branchless sorting network, nearest-edge padding
Savitzky-Golay	Convolution with [-22, 88, 124, 88, -22] / 256	Integer-scaled multiply, edge duplication
SNV	(x - mean) / std	Population std (/ N, not / N-1)

3.3 Verification & Timing Results

On-Device Execution Time: 235 us (measured using CPUTimer0 at 200 MHz)

Verification Comparison:

Comparison	Max Diff	Mean Diff	Match?
preprocessed.csv with ddof=1 vs reference	3.2805	0.3982	No
preprocessed.csv with ddof=0 vs reference	< 1e-6	< 1e-7	Yes

The mismatch was caused by using `np.std(ddof=1)` (sample std) instead of `np.std(ddof=0)` (population std). After fixing, the output matched the reference file to 8 decimal places.

---

4. Machine Learning Model Architecture

The model is a **Hybrid 1D CNN + Linear SVM**: the CNN extracts features, the SVM classifies.

4.1 CNN Backbone Architecture

Layer	Operation	Input Shape	Output Shape
backbone.0	Conv1d	[1, 224]	[64, 28]
backbone.1	BatchNorm1d	[64, 28]	[64, 28]
-	GELU	[64, 28]	[64, 28]
backbone.3	Conv1d	[64, 28]	[128, 28]
backbone.4	BatchNorm1d	[128, 28]	[128, 28]
-	GELU	[128, 28]	[128, 28]
-	MaxPool1d	[128, 28]	[128, 14]
backbone.7	Conv1d	[128, 14]	[128, 14]
backbone.8	BatchNorm1d	[128, 14]	[128, 14]
-	GELU	[128, 14]	[128, 14]
-	GAP	[128, 14]	[128]

4.2 SVM Classification Head

Stage	Operation	Detail
StandardScaler	(x - mean) / std	Fitted on 8,905 training feature vectors
LinearSVC	score[c] = W[c,:] @ scaled + b[c]	5 classes, C=0.5, dual=False
Output	Argmax over 5 scores	Classes are 1-indexed (1 to 5)

Total parameters: ~91,700 Conv weights (int16) + ~1,280 BN params (float32) + 645 SVM params (int16)

---

5. Model Training and Accuracy

5.1 Training Configuration

Parameter	Value
Framework	PyTorch (backbone), scikit-learn (SVM)
Backbone model	nir_cnn_backbone (1).pt
SVM script	retrain_svm.py
SVM type	sklearn.svm.LinearSVC
Regularization C	0.5
Scaler	sklearn.preprocessing.StandardScaler
Training samples for SVM	8,905 preprocessed spectra

5.2 Accuracy at Each Stage

Stage	Accuracy	Notes
LinearSVC on float features (Python)	99.98%	8,905/8,905 training samples
Quantized model simulation (Python)	~99%	Simulates C integer arithmetic
On-device self-test (MCU, 10 samples)	100% (10/10)	Embedded in nir_selftest_data.h

5.3 Per-Class Training Accuracy

Class	Approximate Samples	Python Accuracy
Class 1	~1,781	100.0%
Class 2	~1,781	99.9%
Class 3	~1,781	100.0%
Class 4	~1,781	100.0%
Class 5	~1,781	100.0%

> **Critical note:** The SVM must be re-trained on features extracted from the *current* backbone. Using an SVM from a previous training run causes a systematic class-bias failure (see Bug 3).

---

6. Model-to-Firmware Conversion

The F28379D has no Python runtime. All model weights are compiled as C arrays.

6.1 Conversion Pipeline

```
`PyTorch .pt File
   |
   +-- Conv1d Weights (backbone.0, .3, .7)
   |       |
   |       v  quantize_to_i16()
   |   nir_weights.c / nir_weights.h   (int16_t, split arrays)
   |
   +-- BatchNorm Params (backbone.1, .4, .8)
   |       |
   |       v  kept as raw float32
   |   nir_bn_weights.h
   |
   +-- SVM + Scaler (coef_, intercept_, mean_, scale_)
           |
           v  quantize_to_i16()
       hex files -> embedded in nir_weights.h
`
```

6.2 Quantization Strategy

Parameter Group	Format	Reason
Conv1D weights	int16_t Q16 + per-layer float scale	Halves storage; enables flash sector splits
BatchNorm (gamma, beta, running_mean, running_var)	float32	Quantizing caused catastrophic accuracy loss — see Bug 2
SVM weights + biases	int16_t + scale	640 weights; int16 precision sufficient
StandardScaler mean + std	int16_t + scale	Same

6.3 Flash Memory Banking (C28x Architecture Constraint)

Each C28x flash sector holds a limited number of words. Large arrays must be split:

Array Name	Layer	Elements	Split Strategy
g_bb0_weight	backbone.0	1,600	Single sector
g_bb3_weight_lo + g_bb3_weight_hi	backbone.3	40,960 total	20,480 + 20,480 across two sectors

g_bb7_weight_a + g_bb7_weight_b + g_bb7_weight_c	backbone.7	49,152 total	16,384 + 16,384 + 16,384 across three sectors

---

7. Firmware Implementation

7.1 Hardware Platform

Component	Detail
MCU	TMS320F28379D (Dual-core C28x, 200 MHz each)
Board	Texas Instruments LAUNCHXL-F28379D
Debugger	Onboard XDS110 USB debug probe
UART output	SCI-A TX on GPIO42 -> USB Virtual COM Port
LED	GPIO31 — onboard blue LED D9
Baud rate	115200 baud, 8N1

7.2 CCS Project Structure

Project	Purpose
MCU_Firmware	Production inference firmware (CPU1, self-test mode)
Preprocessing	Standalone C preprocessing pipeline for sensor data
CPU1_MCU_Optimized	Dual-core master (CPU1, controls shared RAM, IPC)
CPU2_MCU_Optimized	Dual-core slave (CPU2, executes parallel sub-tasks)

7.3 Inference Functions in C

Function	Layer	Description
conv1d_stem()	backbone.0	1->64 ch, k=25, stride=8; local weight cache
conv1d_Q16_split()	backbone.3	64->128 ch, reads weights from two flash arrays
conv1d_Q16_split3()	backbone.7	128->128 ch, reads from three arrays; explicit in_stride param
apply_bn_ch()	BN layers	Per-channel BN from float32 arrays in nir_bn_weights.h

gelu_approx()	Activation	Polynomial tanh approximation
maxpool1d_2()	Pooling	In-place, stride 2
run_cnn()	Full backbone	Orchestrates all layers to produce 128-dim feature vector
run_svm()	SVM head	StandardScaler + LinearSVC argmax

7.4 Memory Optimizations

Technique	Implementation
Hot functions in RAM	#pragma CODE_SECTION(run_cnn, ".TI.ramfunc") — zero wait states at 200 MHz
Activation buffers in fast RAM	#pragma DATA_SECTION(g_act0, ".ebss") — GSRAM, 0 wait states
Flash weight splitting	#pragma DATA_SECTION on _lo, _hi, _a, _b, _c sub-arrays
Per-channel weight caching	float w_local[320] preloaded before inner conv loops

7.5 Serial Telemetry Protocol

Message	Format	Example
Boot	BOOT	BOOT
Start	START,,	START,10,5
Per sample	SAMPLE,,,,,,*us*	SAMPLE,3,10,2,2,OK,408us
Final result	RESULT,,,	RESULT,10,10,1000
Confusion matrix	CM,,,,,,	CM,1,2,0,0,0,0
Done	DONE	DONE

7.6 LED Blink Codes

Situation	Pattern
Boot-up	3 slow blinks

Engine ready	5 slow blinks
Per inference	N fast blinks = predicted class N (1-5)
Self-test complete	K long blinks, repeating = K correct out of 10

---

8. Bugs Encountered and How They Were Resolved

Bug Summary Table

#	Symptom	Root Cause	Fix	Outcome
1	No UART output	Wrong SCI peripheral (SCI-B vs SCI-A)	Switch to SCI-A, GPIO42	UART working
2	All = Class 5	BN running_mean quantized to 0 (underflow)	Keep BN as float32	Class diversity restored
3	All = Class 4	SVM/backbone training run mismatch	Re-train SVM on current backbone	Correct predictions
4	Linker error	Weight arrays too large for single flash sector	Split arrays into 2-3 sub-arrays	Build success
5	All = Class 5 (again)	in_stride != in_len after MaxPool	Add in_stride param to conv function	100% accuracy
6	CPU2 not booting	CPU1 symbol defined in CPU2 project	Change to CPU2 in CCS properties	Both cores boot
7	Preprocessing mismatch	SNV used sample std (ddof=1) vs population std (ddof=0)	Change to ddof=0	Exact match

9. Dual-Core Boot Challenges

GEL Memory Map

Both CPU1 and CPU2 show "Memory Map Initialization Complete" in GEL output on debug connect. This is normal — GEL scripts run on both cores at debug session start. It does not mean application code has executed on CPU2.

CPU2 Boot Sequence

CPU2 is held in hardware reset after power-up. CPU1 must explicitly release it:

```
`Device_bootCPU2(C1C2_BROM_BOOTMODE_BOOT_FROM_FLASH);
`
```

CPU2 then executes from flash at `0x080000` (FLASHM sector).

Linker Files

Core	Linker File	Code Sectors Used
CPU1	2837xD_FLASH_lnk_cpu1.cmd	FLASHA, FLASHB, ...
CPU2	2837xD_FLASH_lnk_cpu2.cmd	FLASHM, FLASHN

These sectors do not overlap, ensuring no code collision.

IPC Synchronization

```
`// CPU2: signal ready after Device_init()
IPC_setFlagLtoR(IPC_CPU2_L_CPU1_R, IPC_FLAG_CPU2_READY);

// CPU1: wait before sending inference data
IPC_waitForFlag(IPC_CPU1_L_CPU2_R, IPC_FLAG_CPU2_READY);
`
```

Shared RAM Ownership

CPU1 must grant CPU2 access to shared GSRAM before booting CPU2:

```
`MemCfg_setGSRAMControllerSel(MEMCFG_SECT_GS0, MEMCFG_GSRAMCONTROLLER_CPU2);
MemCfg_setGSRAMControllerSel(MEMCFG_SECT_GS1, MEMCFG_GSRAMCONTROLLER_CPU2);
`
```

---

10. Results Achieved

Final On-Device Performance

Metric	Value
Self-test samples	10
Correct predictions	10 / 10
On-device accuracy	100%
Average preprocessing time	235 us per sample
Average inference time (CNN + SVM)	~400 us per sample
Total cycle time (Prep + Inference)	~635 us per sample

Serial Output (Example)

```
`BOOT
START,10,5
SAMPLE,1,10,3,3,OK,412us
SAMPLE,2,10,1,1,OK,408us
SAMPLE,3,10,5,5,OK,415us
...
RESULT,10,10,1000
CM,1,2,0,0,0,0
CM,2,0,2,0,0,0
CM,3,0,0,2,0,0
CM,4,0,0,0,2,0
CM,5,0,0,0,0,2
AVG_INFERENCE_TIME: 410 us
ACCURACY: 10/10 (100.000%)
DONE
`

![On-Device Serial Execution Telemetry](C:/Users/Yashwanth/.gemini/antigravity-ide/brain/534ea53e-f790-45
```

Confusion Matrix (Self-Test, 10 Samples)

True / Predicted	Class 1	Class 2	Class 3	Class 4	Class 5
Class 1	2	0	0	0	0
Class 2	0	2	0	0	0

Class 3	0	0	2	0	0
Class 4	0	0	0	2	0
Class 5	0	0	0	0	2

11. Future Work: Timing Optimization Plan

Current vs Target

	Current	Target
Pure inference time	400 ms	5 - 15 ms (total with preprocessing)
CPU utilization	CPU1 only	Both CPU1 + CPU2 + 2 CLAs

Proposed Optimizations

Technique	Mechanism	Expected Speedup	Complexity
Dual-core Conv1D split	CPU1 computes ch 0-63, CPU2 computes ch 64-127 in parallel	~1.9x	High (IPC)
CLA co-processing	CLAs run BN + GELU while CPUs run next conv layer	~1.3x	High
DMA ping-pong buffering	Prefetch Flash weights into RAM while GPU computes	~3-4x	Medium
Full .TI.ramfunc	All hot loops run from zero-wait-state RAM	~1.5x	Low
TMU for tanhf() in GELU	Hardware transcendental unit replaces software tanhf	~1.2x	Low
GELU -> ReLU	Eliminates tanhf entirely; requires model retraining	~1.3x	Medium

Dual-Core Workload Split (Proposed)

```
`CPU1 (Master)                          CPU2 (Slave)
-----------------------------------   -----------------------------------
Run stem conv (1->64) -> g_act0       [idle, waiting]
Signal IPC_FLAG_BLOCK1_START       -> Receive signal
Compute Block1, output ch 0..63       Compute Block1, output ch 64..127
Wait for IPC_FLAG_CPU2_BLOCK1_DONE <- Signal done
Merge g_act1 tensor
Run Block2, GAP, SVM
```

```
Output via UART + LED
`
```

---

12. File Inventory

File	Description
MCU/MCU_Firmware/nir_inference_main.c	Main firmware: Conv1D, BN, GELU, GAP, SVM, UART, LED
MCU/MCU_Firmware/nir_weights.c	Split int16_t Conv1D weight arrays
MCU/MCU_Firmware/nir_weights.h	Array declarations and flash sector pragma mappings
MCU/MCU_Firmware/nir_bn_weights.h	BatchNorm parameters in float32 (critical for accuracy)
MCU/MCU_Firmware/nir_model.h	Scale macros, layer dimensions, SG filter coefficients
MCU/MCU_Firmware/nir_selftest_data.h	10 embedded test spectra and ground-truth labels
MCU/Preprocessing/empty_driverlib_main.c	C implementation of the 4-step preprocessing pipeline
MCU/Preprocessing/dark_current_mean.h	224-element mean dark current vector
MCU_Inference/retrain_svm.py	Re-trains SVM on current backbone features
MCU_Inference/generate_weights_c.py	Quantizes Conv1D weights; emits split nir_weights.c
MCU_Inference/generate_weights_bn_float.py	Exports BN float32 params to nir_bn_weights.h
MCU_Inference/generate_selftest_h.py	Generates nir_selftest_data.h from validation spectra
MCU_Inference/quantized_verify.py	C-faithful Python simulation with quantized weights
MCU_Inference/final_verify.py	End-to-end float verification: backbone + new SVM
verify_preprocessing.py	Generates preprocessed.csv from raw data + dark current
SpectrumData_2021Y-trainSet.csv	Raw training dataset (50,000 x 227)
SpectrumData_2021Y-darkcurrent.csv	Dark current reference readings (640 x 224)
MCU_Inference/train_preprocessed_snv.csv	Reference preprocessed training dataset (8,905 x 225)

Report generated from project workspace — June 2026.